XGBRegressor vs Linear Regression

XGBRegressor Overview

XGBRegressor (eXtreme Gradient Boosting Regressor) is an efficient implementation of the gradient boosted decision tree algorithm designed specifically for regression tasks. It is part of the XGBoost library, which is widely used in machine learning due to its speed and high performance.

How it Works

XGBoost is an ensemble method. Instead of fitting a single model to the data, it builds a sequence of simple models (typically decision trees) iteratively:

- The first tree makes an initial rough prediction.
- The second tree focuses on correcting the errors (residuals) of the first tree.
- The third tree focuses on the remaining errors of the previous trees.
- This process continues, with each new tree improving the overall prediction.

Key Differences from Linear Regression

When referring to "normal" regression, it typically means Ordinary Least Squares (OLS) Linear Regression. The key differences are summarized below:

Feature	**Linear Regression (Normal)**	**XGBRegressor**
Relationship	Assumes a linear relationship between inputs and outputs.	Captures complex, non-linear relationships and feature interactions.
Complexity	High bias (simple model); may underfit complex data.	High variance (complex model); may overfit if not tuned properly.
Outliers	Highly sensitive to outliers.	More robust to outliers due to tree-based splitting.
Feature Scaling	Requires feature scaling (e.g., normalization) for optimal performance.	Invariant to feature scaling; works well with raw data.
Handling Missing Data	Cannot handle missing values directly; requires imputation.	Handles missing values automatically using sparsity-aware splitting.
Regularization	Requires separate techniques (Ridge/Lasso) to prevent overfitting.	Includes built-in L1 and L2 regularization.

When to Use Which?

Use Linear Regression if:

- Interpretability is important.
- You need to understand the exact contribution of each feature.
- The relationship between variables is approximately linear.

Use XGBRegressor if:

- The dataset is large and complex.
- There are non-linear relationships and feature interactions.
- The data contains missing values.
- The primary goal is high prediction accuracy.

Mathematical Formulation of XGBRegressor

XGBoost models the prediction as an additive ensemble of decision trees. Given a dataset with n samples and m features, the predicted output for the ith sample is:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(\mathbf{x}_i), \quad f_k \in \mathcal{F} \tag{1}$$

where:

- K is the number of trees,
- f_k represents an individual regression tree,
- $\mathcal{F}$ is the space of all possible trees.

Objective Function

The model is trained by minimizing a regularized objective function:

$$\mathcal{L} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k) \tag{2}$$

where:

- $l(y_i, \hat{y}_i)$ is the loss function (e.g., squared error),
- $\Omega(f_k)$ is the regularization term controlling model complexity.

Regularization Term

The regularization term for a tree is defined as:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \tag{3}$$

where:

- T is the number of leaves in the tree,
- w_j is the weight of leaf j,
- γ controls the complexity (penalty on number of leaves),
- λ is the L2 regularization parameter.

Additive Training (Boosting)

XGBoost builds the model in an additive manner. At iteration t, the prediction is updated as:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i) \tag{4}$$

To optimize efficiently, the loss is approximated using a second-order Taylor expansion:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \right] + \Omega(f_t) \tag{5}$$

where:

$$g_i = \frac{\partial l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}, \quad h_i = \frac{\partial^2 l(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2} \tag{6}$$

Optimal Leaf Weights

For a given tree structure, the optimal weight of leaf j is:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \tag{7}$$

where I_j is the set of samples in leaf j.

Split Gain

The quality of a split is measured using the gain:

$$Gain = \frac{1}{2} \left[\frac{(\sum g_L)^2}{\sum h_L + \lambda} + \frac{(\sum g_R)^2}{\sum h_R + \lambda} - \frac{(\sum g)^2}{\sum h + \lambda} \right] - \gamma \tag{8}$$

where:

- L and R denote left and right child nodes,
- g and h are gradients and Hessians.

Summary

XGBoost improves prediction accuracy by:

- Sequentially adding trees to correct residual errors,
- Using second-order optimization (gradient + Hessian),
- Applying regularization to prevent overfitting,
- Optimizing tree splits using a gain-based criterion.